

React & Next.js

Jour 2 — Composants, State & Hooks



Sommaire — Jour 2

01 Composants fonctionnels & Props

03 Listes, conditions & styling

05 Formulaires contrôlés

07 useEffect & effets de bord

09 Custom Hooks

02 Cycle de vie des composants

04 State & gestion des événements

06 Introduction aux Hooks

08 useRef, useMemo, useCallback

10 TanStack Query & SWR

Rappel Jour 1

- ES6+ : `const` / `let` , arrow functions, destructuration, spread, async/await
- Philosophie React : `UI = f(state)` , composants, déclaratif, Virtual DOM
- JSX : syntaxe compilée en `React.createElement()`
- Vite : initialisation ultra-rapide, HMR natif

Aujourd'hui : on entre dans le cœur de React — state, hooks et gestion des données

| 03 · Les Composants React

Composant Fonctionnel — Définition

```
// Un composant = une fonction JS qui :  
// 1. Reçoit un objet "props" en paramètre  
// 2. Retourne du JSX (ou null)  
  
// Syntaxe avec destructuration des props (recommandé)  
function UserCard({ name, email, role = "Utilisateur", avatar }) {  
  return (  
    <div className="card">  
      <img src={avatar} alt={`Avatar de ${name}`} />  
      <div className="card-body">  
        <h2>{name}</h2>  
        <p>{email}</p>  
        <span className={`badge badge-${role.toLowerCase()} `}>  
          {role}  
        </span>  
      </div>  
    </div>  
  );  
}  
  
export default UserCard;
```



Les Props — Passage de données

```
// Les props sont en lecture seule — JAMAIS les modifier !
function Badge({ color = "blue", size = "md", children }) {
  return (
    <span
      className={`badge badge-${color} badge-${size}`}
    >
      {children}
    </span>
  );
}

// Utilisation — les props se passent comme des attributs HTML
<Badge color="green" size="lg">Actif</Badge>
<Badge color="red">Inactif</Badge>
<Badge>Neutre</Badge>

// Spread des props (attention aux abus)
const buttonProps = { disabled: false, type: "submit" };
<button {...buttonProps}>Envoyer</button>
```

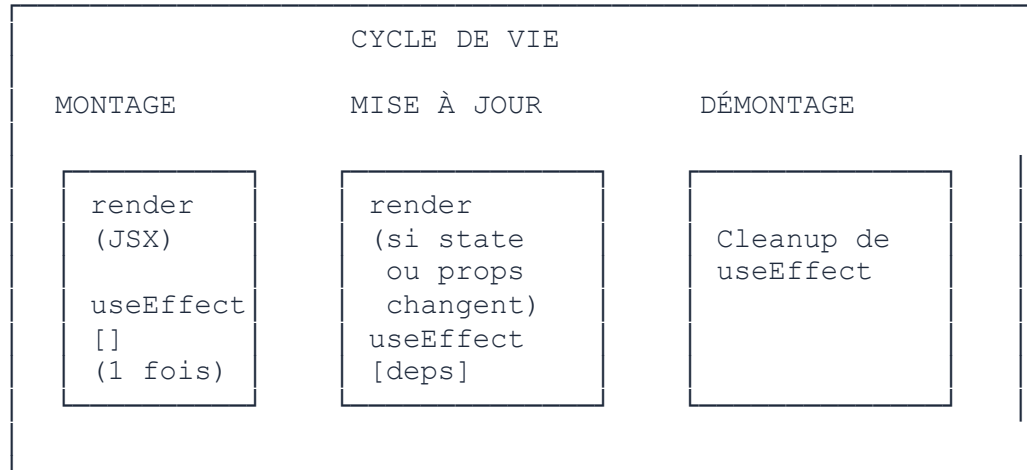


Les Props — Types de données

```
function ProductCard({
  name,           // string
  price,          // number
  inStock,        // boolean
  tags,           // array
  metadata,       // object
  onAddToCart,    // function (callback)
  children,       // ReactNode (JSX)
}) {
  return (
    <div className="product">
      <h3>{name}</h3>
      <p>{price.toFixed(2)} €</p>
      {inStock ? <span>✅ En stock</span> : <span>❌ Épuisé</span>}
      <div>{tags.map(t => <span key={t}>{t}</span>)}</div>
      <button onClick={onAddToCart} disabled={!inStock}>
        Ajouter au panier
      </button>
      <div className="extras">{children}</div>
    </div>
  );
}
```



Cycle de Vie en Composants Fonctionnels



Composants Réutilisables — Design

- **Responsabilité unique** : chaque composant fait une chose bien
- **Props comme API** : définir une interface claire et stable
- **Valeurs par défaut** : toujours prévoir des fallbacks
- `children` : pour les composants conteneurs (layout, card, modal...)
- **Ne pas sur-découper** : un composant trop petit perd en lisibilité

```
// ✅ Bon : composant avec une API claire
<Modal title="Confirmation" onClose={handleClose} isOpen={showModal}>
  <p>Voulez-vous supprimer cet élément ?</p>
  <Button variant="danger" onClick={handleDelete}>Supprimer</Button>
</Modal>
```



La Prop

children

```
// children = ce qui est passé entre les balises ouvrante/fermante
function Card({ title, children }) {
  return (
    <div className="card">
      <div className="card-header">
        <h3>{title}</h3>
      </div>
      <div className="card-body">
        {children} {/* N'importe quel JSX */}
      </div>
    </div>
  );
}

// Utilisation : très flexible !
<Card title="Mon Article">
  <p>Lorem ipsum dolor sit amet...</p>
  
  <Button onClick={handleShare}>Partager</Button>
</Card>
```



Rendu de Listes — Règles des key

```
// ✅ key unique et stable : utiliser l'ID de la donnée
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <span>{todo.title}</span>
          <button onClick={() => onDelete(todo.id)}>X</button>
        </li>
      ))}
    </ul>
  );
}

// ❌ Mauvaises pratiques pour les key
{items.map((item, index) => (
  <li key={index}>...</li> // Dangereux si liste réordonnée
))}

{items.map(item => (
  <li key={Math.random()}>...</li> // Nouveau key à chaque render !
))}
```

? Pourquoi les `key` sont si importantes ?

- La `key` aide React à identifier quel élément a **changé**, été ajouté ou supprimé
- Sans `key` stable, React peut se tromper lors du diff

Liste avant : [A(key=1), B(key=2), C(key=3)]

Liste après : [B(key=2), C(key=3), A(key=1)]

Avec key ID → React déplace les éléments  (3 ops)

Sans key/index → React re-render tout  (9 ops, perte du state local)

La `key` doit être *unique parmi les frères*, *stable* (ne change pas au re-render) et *non liée au contenu affiché*.

Styling — CSS Classique

```
// src/components/Button.jsx

// Option 1 : CSS global importé
import './Button.css';

function Button({ variant = 'primary', children }) {
  return (
    <button className={`btn btn-${variant}`}>
      {children}
    </button>
  );
}

/* Button.css */
.btn { padding: 8px 16px; border-radius: 6px; cursor: pointer; }
.btn-primary { background: #0ea5e9; color: white; border: none; }
.btn-secondary { background: transparent; border: 2px solid #0ea5e9; }
```

Styling — CSS Modules

```
// src/components/Button.module.css
.btn { padding: 8px 16px; border-radius: 6px; cursor: pointer; }
.primary { background: #0ea5e9; color: white; }
.secondary { border: 2px solid #0ea5e9; background: transparent; }

// src/components/Button.jsx
import styles from './Button.module.css';

function Button({ variant = 'primary', children }) {
  return (
    // Les noms de classes sont scopés automatiquement
    // "primary" → "Button_primary__xK2mP" (unique)
    <button className={`${styles.btn} ${styles[variant]}`} >
      {children}
    </button>
  );
}
// ✅ Pas de collision de noms — idéal en équipe
```

Styling — Tailwind CSS

```
// Tailwind : classes utilitaires directement dans le JSX
function Button({ variant = 'primary', disabled, children }) {
  const baseClasses = "px-4 py-2 rounded-md font-medium transition-colors";
  const variantClasses = {
    primary: "bg-sky-500 text-white hover:bg-sky-600",
    secondary: "border-2 border-sky-500 text-sky-500 hover:bg-sky-50",
    danger: "bg-red-500 text-white hover:bg-red-600",
  };

  return (
    <button
      disabled={disabled}
      className={` ${baseClasses} ${variantClasses[variant]}
        ${disabled ? 'opacity-50 cursor-not-allowed' : ''} `
    >
      {children}
    </button>
  );
}
```

Comparatif Approches CSS

Approche	Scoped	TypeScript	Dynamique	Taille bundle	Popularité
CSS global	✗	✗	⚠	Faible	Faible
CSS Modules	✓	✓	⚠	Faible	★ ★ ★
Tailwind CSS	✓	✓	✓	Minimal	★ ★ ★ ★ ★
styled-components	✓	✓	✓	Moyen	★ ★ ★
Emotion	✓	✓	✓	Moyen	★ ★ ★

Recommandation 2025 : Tailwind CSS + shadcn/ui pour les nouveaux projets

| 04 · Gestion de l'État (State)

State vs Props

Props

- Données venant du **parent**
- **En lecture seule**
- Définissent l'**interface** du composant
- Changent quand le parent re-rend
- Passage **top-down**

```
// Le parent contrôle  
<Button disabled={isLoading} />
```

State

- Données **locales** au composant
- **Modifiables** via `setState`
- Déclenchent un **re-render**
- Gérées par le composant lui-même
- **Privées**

```
// Le composant contrôle  
const [open, setOpen] = useState(false);
```



```
import { useState } from 'react';

function Counter() {
  // Déclaration : [valeurCourante, fonctionDeMiseÀJour]
  const [count, setCount] = useState(0); // 0 = valeur initiale

  // Mise à jour : toujours via le setter
  const increment = () => setCount(count + 1);
  const decrement = () => setCount(count - 1);
  const reset      = () => setCount(0);

  return (
    <div>
      <p>Compteur : <strong>{count}</strong></p>
      <button onClick={decrement}>-</button>
      <button onClick={reset}>Reset</button>
      <button onClick={increment}>+</button>
    </div>
  );
}
```



```
function UserForm() {  
  const [user, setUser] = useState({  
    name: "",  
    email: "",  
    age: 0,  
  });  
  
  const updateField = (field, value) => {  
    // ☒ Toujours spread l'objet existant avant de modifier  
    setUser(prev => ({ ...prev, [field]: value }));  
  };  
  
  return (  
    <form>  
      <input  
        value={user.name}  
        onChange={e => updateField('name', e.target.value)}  
      />  
      <input  
        value={user.email}  
        onChange={e => updateField('email', e.target.value)}  
      />  
    </form>  
  );  
}
```



```
function TagList() {
  const [tags, setTags] = useState(["react", "js"]);

  // Ajouter (sans muter)
  const addTag = (tag) => setTags(prev => [...prev, tag]);

  // Supprimer
  const removeTag = (index) =>
    setTags(prev => prev.filter((_, i) => i !== index));

  // Modifier
  const updateTag = (index, newTag) =>
    setTags(prev => prev.map((t, i) => i === index ? newTag : t));

  // Réordonner
  const shuffle = () =>
    setTags(prev => [...prev].sort(() => Math.random() - 0.5));

  return (
    <div>
      {tags.map((tag, i) => (
        <span key={tag}>{tag} <button onClick={() => removeTag(i)}>x</button></span>
      ))}
    </div>
  );
}
```



— Mise à jour Fonctionnelle

```
// ⚠ Piège : state mis à jour de façon asynchrone !  
const [count, setCount] = useState(0);  
  
// ❌ Problème : plusieurs appels en rafale  
function handleTripleClick() {  
  setCount(count + 1); // count est toujours 0 ici !  
  setCount(count + 1); // count est toujours 0 !  
  setCount(count + 1); // count est toujours 0 !  
  // Résultat : count = 1, pas 3 !  
}  
  
// ✅ Solution : forme fonctionnelle (reçoit la valeur la plus récente)  
function handleTripleClick() {  
  setCount(prev => prev + 1); // 0 → 1  
  setCount(prev => prev + 1); // 1 → 2  
  setCount(prev => prev + 1); // 2 → 3  
  // Résultat : count = 3 ✓  
}
```



Gestion des Événements

```
function EventDemo() {
  const handleClick = (e) => {
    console.log("Cliqué !", e.target);
    e.preventDefault(); // Pour les liens/formulaires
    e.stopPropagation(); // Stop la propagation
  };

  const handleKeyDown = (e) => {
    if (e.key === 'Enter') console.log("Entrée pressée");
    if (e.ctrlKey && e.key === 's') console.log("Ctrl+S");
  };

  const handleMouseEnter = () => console.log("Hover !");

  return (
    <div
      onClick={handleClick}
      onKeyDown={handleKeyDown}
      onMouseEnter={handleMouseEnter}
    >
      { /* Passer des arguments à un handler */ }
      <button onClick={e => handleDelete(item.id, e)}>
        Supprimer
      </button>
    </div>
  );
}
```



Formulaires — Input Contrôlé

```
// Input contrôlé : la valeur est toujours pilotée par le state
function SearchBar() {
  const [query, setQuery] = useState('');

  const handleChange = (e) => {
    setQuery(e.target.value);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Recherche :", query);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={query}           // ← Piloté par le state
        onChange={handleChange}
        placeholder="Rechercher..."
      />
      <button type="submit">🔍</button>
      <p>Requête : {query}</p>
    </form>
  );
}
```




Formulaires — Validation

```
function LoginForm() {
  const [form, setForm] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!form.email.includes('@'))
      newErrors.email = "Email invalide";
    if (form.password.length < 8)
      newErrors.password = "Minimum 8 caractères";
    return newErrors;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    if (Object.keys(errs).length > 0) { setErrors(errs); return; }
    // Soumettre le formulaire
    console.log("Connexion :", form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={form.email} onChange={e => setForm({...form, email: e.target.value})} />
      {errors.email && <p className="error">{errors.email}</p>}
    </form>
  );
}
```



Formulaires — Select, Checkbox, Radio

```
function PreferenceForm() {
  const [prefs, setPrefs] = useState({
    country: 'fr',
    newsletter: true,
    plan: 'pro',
  });

  const update = (field) => (e) => {
    const value = e.target.type === 'checkbox'
      ? e.target.checked
      : e.target.value;
    setPrefs(p => ({ ...p, [field]: value }));
  };

  return (
    <form>
      <select value={prefs.country} onChange={update('country')}>
        <option value="fr">France</option>
        <option value="be">Belgique</option>
      </select>
      <input type="checkbox" checked={prefs.newsletter} onChange={update('newsletter')} />
      <input type="radio" value="free" checked={prefs.plan === 'free'} onChange={update('plan')} />
      <input type="radio" value="pro" checked={prefs.plan === 'pro'} onChange={update('plan')} />
    </form>
  );
}
```

Lifting State Up

```
// Remonter le state vers l'ancêtre commun des composants qui en ont besoin

function ParentSearch() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const handleSearch = async (q) => {
    setQuery(q);
    const data = await searchAPI(q);
    setResults(data);
  };

  return (
    <div>
      {/* SearchBar reçoit le handler */}
      <SearchBar onSearch={handleSearch} value={query} />
      {/* ResultList reçoit les résultats */}
      <ResultList items={results} />
    </div>
  );
}

// Règle : minimiser le state – le placer au plus bas niveau possible
// qui le partage entre plusieurs enfants
```



Créer une **Todo List** complète avec :

- +
- 
- 
- 
- 
- 



```
// State initial
const [todos, setTodos] = useState([
  { id: 1, title: "Apprendre React", done: false },
  { id: 2, title: "Créer un composant", done: true },
  { id: 3, title: "Maîtriser les hooks", done: false },
]);

const [filter, setFilter] = useState('all'); // 'all' | 'active' | 'done'
const [newTitle, setNewTitle] = useState('');

// Données calculées (dérivées du state)
const filteredTodos = todos.filter(todo => {
  if (filter === 'active') return !todo.done;
  if (filter === 'done') return todo.done;
  return true;
});

const remaining = todos.filter(t => !t.done).length;
```



```
// Ajouter
const addTodo = (e) => {
  e.preventDefault();
  if (!newTitle.trim()) return;
  setTodos(prev => [
    ...prev,
    { id: Date.now(), title: newTitle.trim(), done: false }
  ]);
  setNewTitle('');
};

// Toggle done
const toggleTodo = (id) =>
  setTodos(prev => prev.map(t =>
    t.id === id ? { ...t, done: !t.done } : t
  ));

// Supprimer
const deleteTodo = (id) =>
  setTodos(prev => prev.filter(t => t.id !== id));

// Modifier
const editTodo = (id, newTitle) =>
  setTodos(prev => prev.map(t =>
    t.id === id ? { ...t, title: newTitle } : t
  ));
```

| 05 · Les Hooks React

Pourquoi les Hooks ?

- **Avant React 16.8** : seuls les class components avaient un state
- Les class components : `this`, lifecycle methods verbeux, difficiles à réutiliser
- **Hooks** (16.8, 2019) : state et effets dans les composants fonctionnels
- **Avantages** :
 - Partager de la logique entre composants (custom hooks)
 - Code plus concis et lisible
 - Meilleure testabilité
 - Pas de `this`

Les hooks ont révolutionné l'écriture de React. Toute la codebase moderne utilise des hooks.

Les Hooks Natifs de React

- `useState` — state local
- `useEffect` — effets de bord
- `useRef` — référence mutable
- `useMemo` — mémoïsation de valeur
- `useCallback` — mémoïsation de fonction
- `useContext` — consommer un contexte

- `useReducer` — state complexe
- `useLayoutEffect` — après le DOM
- `useImperativeHandle` — refs custom
- `useId` — identifiants uniques
- `useTransition` — React 18+
- `useDeferredValue` — React 18+

Règles des Hooks (à ne jamais violer)

- 1. Ne pas appeler dans des conditions

```
// ❌ INTERDIT
if (condition) {
  const [state, setState] = useState(0); // Erreur !
}

// ✅ Toujours au niveau racine du composant
const [state, setState] = useState(0);
if (condition) { /* utiliser state ici */ }
```

- 2. Ne pas appeler dans des fonctions ordinaires

```
// ✅ Dans un composant React ou un custom hook uniquement
function MyComponent() { const [x] = useState(0); }
function useMyHook() { const [x] = useState(0); } // custom hook
// ❌ function helper() { const [x] = useState(0); } → interdit
```



```
import { useState, useEffect } from 'react';

function PageTitle({ title }) {
  // useEffect s'exécute après chaque render
  useEffect(() => {
    // Code à exécuter : effets de bord
    document.title = `${title} | Mon App`;

    // Optionnel : fonction de cleanup
    return () => {
      document.title = "Mon App";
    };
  }, [title]); // Tableau de dépendances

  return <h1>{title}</h1>;
}
```



```
// Cas 1 : Aucune dépendance → s'exécute à CHAQUE render
useEffect(() => {
  console.log("Après chaque render");
});

// Cas 2 : Tableau vide [] → s'exécute UNE SEULE FOIS (montage)
useEffect(() => {
  console.log("Au montage uniquement");
  return () => console.log("Au démontage");
}, []);

// Cas 3 : Avec dépendances → s'exécute quand une dép change
useEffect(() => {
  console.log("userId a changé :", userId);
  // fetch les données pour ce userId
}, [userId]);

// ⚠ Règle : toujours lister toutes les valeurs réactives utilisées
// (eslint-plugin-react-hooks vous aide à les détecter)
```



```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false; // Éviter les race conditions

    async function fetchUser() {
      try {
        setLoading(true);
        const res = await fetch(`/api/users/${userId}`);
        if (!res.ok) throw new Error('Erreur réseau');
        const data = await res.json();
        if (!cancelled) setUser(data);
      } catch (err) {
        if (!cancelled) setError(err.message);
      } finally {
        if (!cancelled) setLoading(false);
      }
    }

    fetchUser();
    return () => { cancelled = true; }; // Cleanup
  }, [userId]);
```



```
function WindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    };

    // Abonnement
    window.addEventListener('resize', handleResize);

    // Cleanup : désabonnement indispensable pour éviter les fuites mémoire
    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []); // Tableau vide : on s'abonne une seule fois

  return <p>{size.width} × {size.height}</p>;
}
```



```
// ❌ Anti-pattern 1 : manquer des dépendances
useEffect(() => {
  setCount(count + 1); // count est une dépendance !
}, []); // ESLint vous avertit → react-hooks/exhaustive-deps

// ❌ Anti-pattern 2 : objet/fonction créé pendant le render
useEffect(() => {
  fetch(config.url); // config est recréé à chaque render !
}, [config]); // Loop infinie !

// ✅ Solution : mémoriser config avec useMemo ou le mettre hors du composant

// ❌ Anti-pattern 3 : useEffect inutile pour du calcul
// const total = useEffect(() => { setTotal(items.length); }, [items]);
// ✅ Solution : calcul directement pendant le render
const total = items.length; // Pas besoin d'useEffect !
```



```
import { useRef } from 'react';

// Usage 1 : Accéder à un élément DOM
function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus();
    inputRef.current.select();
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Votre texte..." />
      <button onClick={handleFocus}>Focus 🎯</button>
    </div>
  );
}

// Différence avec useState :
// useRef ne déclenche PAS de re-render quand .current change
```




```
// Usage 2 : Stocker une valeur qui persiste entre les renders
// sans déclencher de re-render

function Timer() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null); // Stocker l'ID de l'interval

  const start = () => {
    if (intervalRef.current) return; // Déjà démarré
    intervalRef.current = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);
  };

  const stop = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  };

  useEffect(() => () => stop(), []); // Cleanup au démontage

  return <div><p>{seconds}s</p><button onClick={start}>▶</button><button onClick={stop}>◻</button></div>;
}
```



```
import { useMemo } from 'react';

function ExpensiveList({ items, filter }) {
  // Sans useMemo : recalculé à CHAQUE render du composant
  // const filtered = items.filter(item => item.name.includes(filter));

  // Avec useMemo : recalculé UNIQUEMENT si items ou filter changent
  const filteredItems = useMemo(() => {
    console.log("Calcul en cours..."); // Pour démontrer
    return items
      .filter(item => item.name.toLowerCase().includes(filter.toLowerCase()))
      .sort((a, b) => a.name.localeCompare(b.name));
  }, [items, filter]); // ← Dépendances

  return <ul>{filteredItems.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
}

// ⚠ N'optimisez pas prématurément ! useMemo a un coût.
// À utiliser seulement pour des calculs vraiment coûteux.
```



```
import { useState, useCallback, memo } from 'react';

// memo : re-render seulement si les props changent
const Button = memo(({ onClick, label }) => {
  console.log(`Render : ${label}`);
  return <button onClick={onClick}>{label}</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // ❌ Sans useCallback : nouvelle référence à chaque render
  // const handleClick = () => console.log("Cliqué");

  // ✅ Avec useCallback : même référence si les dép ne changent pas
  const handleClick = useCallback(() => {
    console.log("Cliqué, count :", count);
  }, [count]); // Nouvelle fonction seulement si count change

  return <Button onClick={handleClick} label="Cliquer" />;
}
```



Quand Optimiser ?

⚠ Ne pas sur-optimiser

- `useMemo` / `useCallback` ont un coût en mémoire
- Code plus complexe et verbeux
- **Profiler d'abord**, optimiser ensuite
- React est déjà très rapide pour la plupart des cas

✅ Optimiser quand...

- Un calcul prend **>1ms** et se répète
- Un composant enfant est `memo()` et reçoit une fonction
- Liste de **+1000 éléments** avec transformations
- Jank visible dans le profiler React DevTools

| 06 · Custom Hooks

Custom Hooks — Concept

```
// Un custom hook = une fonction JS qui commence par "use"
// et peut appeler d'autres hooks

// ✅ Extraction de logique commune
// ❌ Avant : logique de fetch dupliquée dans chaque composant
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => { /* fetch users... */ }, []);
  // ...
}

function PostList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => { /* fetch posts... */ }, []);
  // ...
}

// ✅ Après : hook réutilisable
const { data: users, loading } = useFetch('/api/users');
const { data: posts, loading } = useFetch('/api/posts');
```

Custom Hook — useFetch

```
// src/hooks/useFetch.js

import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    setError(null);

    fetch(url)
      .then(r => r.ok ? r.json() : Promise.reject(r.status))
      .then(d => { if (!cancelled) setData(d); })
      .catch(e => { if (!cancelled) setError(e); })
      .finally(() => { if (!cancelled) setLoading(false); });

    return () => { cancelled = true; };
  }, [url]);

  return { data, loading, error };
}

export default useFetch;
```

Custom Hook — `useLocalStorage`

```
// src/hooks/useLocalStorage.js

import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Utilisation - identique à useState !
const [theme, setTheme] = useLocalStorage('theme', 'light');
const [cart, setCart] = useLocalStorage('cart', []);
```


🔗 Custom Hook — `useDebounce`

```
// src/hooks/useDebounce.js
import { useState, useEffect } from 'react';

function useDebounce(value, delay = 300) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer); // Cleanup à chaque changement
  }, [value, delay]);

  return debouncedValue;
}

// Utilisation : recherche avec debounce
function SearchBar() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 400);

  useEffect(() => {
    if (debouncedQuery) fetchResults(debouncedQuery);
  }, [debouncedQuery]); // Déclenché seulement 400ms après la dernière frappe
}
```



Custom Hook — `useMediaQuery`

```
// src/hooks/useMediaQuery.js
import { useState, useEffect } from 'react';

function useMediaQuery(query) {
  const [matches, setMatches] = useState(
    () => window.matchMedia(query).matches
  );

  useEffect(() => {
    const mq = window.matchMedia(query);
    const handler = (e) => setMatches(e.matches);
    mq.addEventListener('change', handler);
    return () => mq.removeEventListener('change', handler);
  }, [query]);

  return matches;
}

// Utilisation - responsive en JS
function ResponsiveNav() {
  const isMobile = useMediaQuery('(max-width: 768px)');
  return isMobile ? <HamburgerMenu /> : <DesktopNav />;
}
```

| 07 · TanStack Query & SWR

Pourquoi TanStack Query ?

Problèmes avec `useEffect` + `fetch` :

- Pas de cache → re-fetch inutiles à chaque navigation
- Pas de deduplication → requêtes dupliquées
- Gestion manuelle du `loading` / `error`
- Pas de refetch automatique (focus, réseau)
- Pas de pagination, infinite scroll, mutations...

TanStack Query (ex-React Query) apporte :

- **Cache intelligent** avec stale time configurable
- **Deduplication** des requêtes identiques
- **Refetch automatique** (window focus, reconnexion réseau)
- **Optimistic updates**, mutations, invalidation de cache

TanStack Query — Setup

```
npm install @tanstack/react-query
```

```
// src/main.jsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // 5 minutes
      retry: 2,
    },
  },
});

createRoot(document.getElementById('root')).render(
  <QueryClientProvider client={queryClient}>
    <App />
    <ReactQueryDevtools initialIsOpen={false} />
  </QueryClientProvider>
);
```

```
import { useQuery } from '@tanstack/react-query';

async function fetchMovies() {
  const res = await fetch('https://api.themoviedb.org/3/movie/popular');
  if (!res.ok) throw new Error('Erreur API');
  return res.json();
}

function MovieList() {
  const {
    data,
    isLoading,
    isError,
    error,
    refetch,
    isFetching,
  } = useQuery({
    queryKey: ['movies', 'popular'], // Clé de cache unique
    queryFn: fetchMovies,
    staleTime: 10 * 60 * 1000,      // 10min avant re-fetch
  });

  if (isLoading) return <p>Chargement...</p>;
  if (isError) return <p>Erreur : {error.message}</p>;
  return <ul>{data.results.map(m => <li key={m.id}>{m.title}</li>)}</ul>;
}
```

TanStack Query — `useQuery` avec Paramètres

```
function MovieDetail({ movieId }) {
  const { data: movie } = useQuery({
    queryKey: ['movie', movieId], // La clé change avec movieId
    queryFn: () =>
      fetch(`https://api.themoviedb.org/3/movie/${movieId}`)
        .then(r => r.json()),
    enabled: !!movieId, // Désactive la requête si movieId est falsy
    staleTime: 30 * 60 * 1000, // Les détails restent frais 30min
  });

  // Prefetching au hover (UX pro)
  const handleHover = () => {
    queryClient.prefetchQuery({
      queryKey: ['movie', movieId],
      queryFn: () => fetch(`.../${movieId}`).then(r => r.json()),
    });
  };

  return <div onMouseEnter={handleHover}>{movie?.title}</div>;
}
```



```
import { useMutation, useQueryClient } from '@tanstack/react-query';

function AddMovieForm() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (newMovie) =>
      fetch('/api/movies', {
        method: 'POST',
        body: JSON.stringify(newMovie),
        headers: { 'Content-Type': 'application/json' },
      }).then(r => r.json()),

    onSuccess: () => {
      // Invalider le cache pour forcer un refetch
      queryClient.invalidateQueries({ queryKey: ['movies'] });
    },
    onError: (err) => console.error("Erreur :", err),
  });

  return (
    <button
      onClick={() => mutation.mutate({ title: "Nouveau Film" })}
      disabled={mutation.isPending}
    >
      {mutation.isPending ? "Envoi..." : "Ajouter"}
    </button>
  );
}
```




TanStack Query vs SWR vs [useEffect](#)

Critère	useEffect	SWR	TanStack Query
Cache	✗	✓	✓
Deduplication	✗	✓	✓
Refetch auto	✗	✓	✓
Mutations	✗	⚠	✓
Optimistic updates	✗	⚠	✓
Pagination / Infinite	✗	⚠	✓
DevTools	✗	✗	✓
Taille bundle	—	~4ko	~13ko
Courbe apprentissage	Faible	Faible	Moyenne



Construire un catalogue avec TanStack Query :

- A small icon of a clapperboard, representing media or video content.
- A small icon of a magnifying glass, representing search or discovery.
- A small icon of a document with lines, representing text or metadata.
- A small icon of a video camera, representing video content.
- A small orange star icon, representing favorites or recommendations.



```
npm install @tanstack/react-query @tanstack/react-query-devtools
```

```
// Clé API TMDB (gratuite sur themoviedb.org)
// À placer dans .env.local
VITE_TMDB_API_KEY=votre_clé_ici
VITE_TMDB_BASE_URL=https://api.themoviedb.org/3

// src/api/movies.js
const BASE = import.meta.env.VITE_TMDB_BASE_URL;
const KEY = import.meta.env.VITE_TMDB_API_KEY;

export const getPopularMovies = (page = 1) =>
  fetch(`${BASE}/movie/popular?api_key=${KEY}&language=fr-FR&page=${page}`)
    .then(r => r.json());

export const searchMovies = (query, page = 1) =>
  fetch(`${BASE}/search/movie?api_key=${KEY}&query=${query}&page=${page}`)
    .then(r => r.json());
```



```
// src/hooks/useMovies.js
import { useQuery } from '@tanstack/react-query';
import { getPopularMovies, searchMovies } from '../api/movies';

export function usePopularMovies(page = 1) {
  return useQuery({
    queryKey: ['movies', 'popular', page],
    queryFn: () => getPopularMovies(page),
    staleTime: 10 * 60 * 1000,
    keepPreviousData: true, // Garde l'ancienne page pendant le chargement
  });
}

export function useSearchMovies(query, page = 1) {
  return useQuery({
    queryKey: ['movies', 'search', query, page],
    queryFn: () => searchMovies(query, page),
    enabled: query.length >= 2, // Recherche à partir de 2 caractères
    staleTime: 5 * 60 * 1000,
  });
}
```



```
// src/components/MovieCard.jsx
function MovieCard({ movie, isFavorite, onToggleFavorite }) {
  const imgUrl = movie.poster_path
    ? `https://image.tmdb.org/t/p/w300${movie.poster_path}`
    : '/placeholder.jpg';

  return (
    <div className="movie-card">
      <img src={imgUrl} alt={movie.title} />
      <div className="movie-info">
        <h3>{movie.title}</h3>
        <p>★ {movie.vote_average.toFixed(1)} / 10</p>
        <p>📅 {movie.release_date?.slice(0, 4)}</p>
      </div>
      <button
        className={`fav-btn ${isFavorite ? 'active' : ''}`}
        onClick={() => onToggleFavorite(movie.id)}
      >
        {isFavorite ? '❤️' : '💖'}
      </button>
    </div>
  );
}
```



Quiz Jour 2 — Question 1

Quelle est la bonne façon d'ajouter un item à un state tableau ?

```
const [items, setItems] = useState(['a', 'b']);
```

- A) `items.push('c'); setItems(items);`
- B) `setItems([...items, 'c'])` ✓
- C) `setItems(items.concat = ['c'])`
- D) `setItems(items + ['c'])`

Ne jamais **muter** le state directement. Toujours créer un nouveau tableau avec spread ou `.concat()`.



Quiz Jour 2 — Question 2

Quand `useEffect(() => { ... }, [])` s'exécute-t-il ?

- A) À chaque render du composant
- B) Uniquement quand le state change
- C) Une seule fois, au montage du composant ✓
- D) Jamais

Un tableau de dépendances **vide** `[]` signifie "aucune dépendance → ne se réexécute jamais après le premier render".



Quiz Jour 2 — Question 3

Quelle est la principale différence entre `useRef` et `useState` ?

- A) `useRef` ne peut stocker que des éléments DOM
- B) `useState` provoque un re-render, `useRef` non ✓
- C) `useRef` est plus performant pour les chaînes de caractères
- D) Il n'y a aucune différence

`useRef.current` peut changer sans déclencher de re-render — idéal pour stocker des valeurs "internes" comme des timers ou des positions de scroll.



Quiz Jour 2 — Question 4

Pourquoi TanStack Query est préférable à `useEffect` pour les appels API ?

- A) Il est plus simple à écrire
- B) Il fournit un cache, de la deduplication et du refetch automatique ✓
- C) Il évite d'écrire du JavaScript asynchrone
- D) Il est intégré nativement dans React

TanStack Query gère automatiquement des patterns complexes (cache, retry, stale-while-revalidate) qu'il faudrait implémenter manuellement avec `useEffect` .



Quiz Jour 2 — Question 5

Laquelle de ces règles des hooks est incorrecte ?

- A) Ne pas appeler les hooks dans des conditions `if`
- B) Ne pas appeler les hooks dans des boucles `for`
- C) Les custom hooks doivent commencer par `use`
- D) Les hooks ne peuvent être appelés que dans les class components ☒

Les hooks sont **exclusivement** pour les composants **fonctionnels** (et les custom hooks). Les class components utilisent `this.state` et les méthodes lifecycle.

Récapitulatif Jour 2

-  **Composants** : fonctionnels, props typées, `children`, réutilisables
-  **Listes & keys** : `map()`, keys stables et uniques
-  **CSS** : Modules, Tailwind, CSS-in-JS — Tailwind recommandé 2025
-  **useState** : state local, mises à jour immuables, forme fonctionnelle
-  **Formulaires** : inputs contrôlés, validation, lifting state up
-  **useEffect** : effets de bord, dépendances, cleanup obligatoire
-  **useRef** : DOM et valeurs persistantes sans re-render
-  **Custom hooks** : extraction et réutilisation de logique
-  **TanStack Query** : cache, mutations, invalidation — standard 2025

Pour Aller Plus Loin

- TanStack Query docs : tanstack.com/query/latest
- React Patterns : reactpatterns.com
- useHooks : usehooks.com (bibliothèque de hooks)
- React DevTools : Profiler onglet pour analyser les renders

Exercice maison : Ajoutez de la pagination au catalogue de films avec `keepPreviousData: true` pour une UX fluide entre les pages.

Fin du Jour 2 🎉